

Joc del Pingu — Documentació CRUD

Anàlisi detallat de totes les sentències SQL embegudes al codi, amb captures de pantalla i context de cada operació

Grup DW25-26 GR06 — Curs 2025/2026

Índex

1. Introducció i visió general
 - 1.1. Què documenta aquest informe
 - 1.2. Arquitectura de la capa de persistència
 - 1.3. Connexió a la base de dades Oracle
 - 1.4. Encriptació prèvia (CryptoUtil)
2. Esquema de la base de dades
 - 2.1. Taula ENTITY
 - 2.2. Taula SAVED_GAMES
 - 2.3. Taula GAME
 - 2.4. Taula BOARD
3. Operacions INSERT
 - 3.1. INSERT al panell de proves CRUD
 - 3.2. INSERT (via MERGE) al registre de jugador
 - 3.3. INSERT d'una partida guardada (CLOB)
 - 3.4. INSERT a la taula GAME (final de partida)
 - 3.5. INSERT idempotent a la taula BOARD
4. Operacions UPDATE
 - 4.1. UPDATE de contrasenya (panell de proves)
 - 4.2. Increment de GAMES_PLAYED
 - 4.3. Increment de GAMES_WON
 - 4.4. UPDATE de l'estat de la partida a FINISHED
 - 4.5. UPDATE (via MERGE) en actualitzar un jugador existent
5. Operacions DELETE
 - 5.1. DELETE al panell de proves
 - 5.2. Wrapper genèric delete()
6. Operacions SELECT
 - 6.1. SELECT amb mètode print() (panell de proves)
 - 6.2. Llistat de partides guardades
 - 6.3. Carregar una partida concreta
 - 6.4. Verificació de contrasenya
 - 6.5. Llistat de jugadors registrats
 - 6.6. Estadístiques / leaderboard
 - 6.7. Càlcul del següent ENTITYID
 - 6.8. Comprovació d'existència (subquery NOT EXISTS)
7. Operacions especials
 - 7.1. MERGE (upsert atòmic)

- 7.2. PreparedStatement i el límit ORA-01704
 - 7.3. Trigger TRG_GAME_FINISHED
 - 7.4. Restricció CK_PLAYER_COLOUR
 - 7.5. NVL i la propagació de NULL a Oracle
8. Panell de proves CRUD (BBDDPanel)
9. Resum de fixters i ubicacions SQL

1. Introducció i visió general

1.1. Què documenta aquest informe

Aquest document recull **tota la part del Joc del Pingu on s'embegen sentències SQL**. Per a cada operació es proporciona:

- La **ubicació exacta** al codi font (fitxer i línia).
- La **sentència SQL** tal com s'envia a Oracle.
- El **context funcional**: quin botó o flux del joc l'activa.
- Una **captura de pantalla** que mostra el moment a la UI o el resultat a la BBDD.
- **Justificacions tècniques** de les decisions preses (per què MERGE i no INSERT, per què PreparedStatement, per què NVL, etc.).

L'objectiu és que un docent o un altre desenvolupador pugui validar ràpidament que el joc compleix els requisits CRUD del projecte, sense haver de navegar pel codi.

1.2. Arquitectura de la capa de persistència

L'aplicació segueix una arquitectura en capes per a tot el que toca la base de dades. De més baix nivell a més alt:

1. **model.db.BBDD**: facade genèrica de JDBC. Exposa `insert()`, `update()`, `delete()`, `select()`, `print()` i `executeInsUpDel()`. Tots aquests mètodes accepten una sentència SQL ja construïda i un objecte `Connection`. No coneix res del joc.
2. **model.game.SaveLoadService**: facade específica del domini "Joc del Pingu". Construeix les sentències SQL concretes per a cada cas d'ús (registre de jugador, save game, load game, leaderboard, etc.) i les passa als mètodes de BBDD.
3. **Controladors de la UI** (`MainMenuController`, `PlayerSetupController`, `PlayerStatsController`, `GameBoardController`): criden els mètodes d'alt nivell de SaveLoadService quan l'usuari interacciona amb la UI. Mai construeixen SQL directament.
4. **view.ui.BBDDPanel**: classe separada amb el seu propi `main`, utilitzada exclusivament per al lliurament inicial del projecte. Permet provar les quatre operacions CRUD bàsiques contra la taula `ENTITY` sense haver d'iniciar el joc complet.

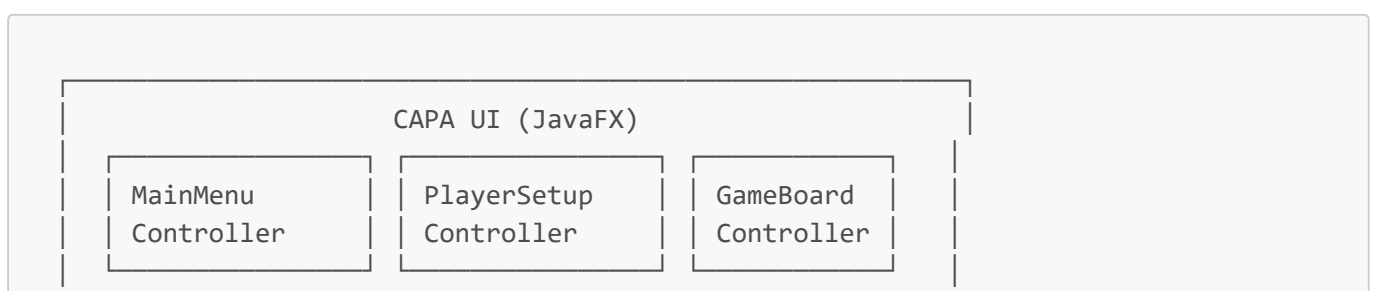




Figura 1.1: Diagrama de capes de la persistència. Les crides flueixen sempre de dalt cap a baix.

1.3. Connexió a la base de dades Oracle

Ubicació: `src/model/db/BBDD.java` — línies 23-59

Tot accés a la BBDD comença amb l'obtenció d'un objecte `Connection` mitjançant el mètode estàtic `conectarBaseDatos(Scanner)`. La signatura accepta un `Scanner` per compatibilitat amb una versió anterior que demanava les credencials per consola, però l'usuari i contrasenya estan **hardcoded** per simplificar el desplegament durant el curs.

El mètode realitza tres operacions clau:

1. **Construeix la URL JDBC** en funció d'una variable interna `entorno` que pot prendre dos valors:

- `"centro"` → IP local del servidor a Ilerna.
- `"fuera"` → DNS públic `oracle.ilerna.com`.

Permet treballar tant des de les aules com des de casa sense modificar res més.

2. **Carrega el driver d'Oracle** via `Class.forName("oracle.jdbc.driver.OracleDriver")`. Si el JAR no està al classpath, salta una `ClassNotFoundException` amb un missatge clar.
3. **Estableix la connexió** amb `DriverManager.getConnection(url, user, pwd)` i la valida amb `con.isValid(5)` (timeout de 5 segons).

```
public static Connection conectarBaseDatos(Scanner scan) {
    System.out.println("Intentando conectarse a la base de datos...");
    String entorno = "fuera"; // canvia a "centro" si treballes des d'Ilerna
    String url = entorno.equals("centro")
        ? "jdbc:oracle:thin:@//192.168.3.26:1521/XEPDB2"
        : "jdbc:oracle:thin:@//oracle.ilerna.com:1521/XEPDB2";

    // Credenciales hardcoded
    String user = "DW2526_GR06_PINGU";
    String pwd = "ABDJFMV";

    try {
        Class.forName("oracle.jdbc.driver.OracleDriver");
        Connection con = DriverManager.getConnection(url, user, pwd);
        if (con.isValid(5)) {
            System.out.println("Database connection established successfully.");
        }
        return con;
    } catch (ClassNotFoundException e) {
        System.err.println("Oracle JDBC driver not found.");
    } catch (SQLException e) {
        System.err.println("Failed to connect to database at: " + url);
    }
    return null;
}
```

⚠️ Avis de seguretat: les credencials reals estan al codi font però en aquesta documentació apareixen com a placeholders. Per a producció caldria moure-les a un fitxer de configuració extern, variables d'entorn o un keystore. En un entorn acadèmic com aquest s'accepta el hardcoding pel context i la curta vida de les credencials.

La política d'errors és **retornar null en lloc de llançar excepcions**. Aquesta decisió simplifica el codi superior: els mètodes de `SaveLoadService` només han de comprovar si la connexió és `null` i, en aquest cas, retornar una llista buida o `false`, de manera que la UI mostra missatges amigables i no un stack trace.

[Captura 1.2] Sortida per consola del programa en iniciar-se, mostrant les dues línies de confirmació:

```
Intentando conectarse a la base de datos...
Database connection established successfully.
```

1.4. Encriptació prèvia (CryptoUtil)

Ubicació: `src/model/config/CryptoUtil.java`

Abans de fer un INSERT o UPDATE amb dades sensibles (contrasenyes, partides guardades), aquestes es passen per `CryptoUtil.encrypt()`. La classe utilitza **AES-128 en mode ECB + PKCS5Padding** amb una clau fixa de 16 bytes ("`PinguGameKey1234`") i retorna el ciphertext codificat en **Base64**.

L'ús de Base64 és deliberat: permet emmagatzemar el resultat com a text en columnes `VARCHAR2` / `CLOB` sense problemes d'escapat ni de bytes binaris.

```
public static String encrypt(String value) {
    SecretKeySpec secretKey = new SecretKeySpec(KEY, ALGORITHM);
    Cipher cipher = Cipher.getInstance(ALGORITHM);
    cipher.init(Cipher.ENCRYPT_MODE, secretKey);
    byte[] encryptedBytes = cipher.doFinal(value.getBytes("UTF-8"));
    return Base64.getEncoder().encodeToString(encryptedBytes);
}
```

✦ **Per què AES amb clau fixa i ECB?** És obfuscatió, no seguretat criptogràfica. L'objectiu és que un usuari que obri SQL Developer directament no vegi les contrasenyes en clar. Per a un sistema real caldria un mode autenticat com GCM, una clau derivada de l'usuari (PBKDF2) i mai hardcoded.

2. Esquema de la base de dades

El joc interactua amb quatre taules principals. Aquest esquema es va dissenyar al lliurament de modelat de la BBDD i no es modifica des de Java; les sentències DDL (`CREATE TABLE`, triggers, restriccions) es van executar manualment al servidor Oracle.

2.1. Taula `ENTITY`

Conté **tots els participants registrats** del joc: jugadors humans i, potencialment, la foca. Cada fila representa una identitat persistent.

Columna	Tipus	NULL?	Propòsit
ENTITYID	NUMBER (PK)	NOT NULL	Identificador únic. Es genera com <code>MAX(ENTITYID) + 1</code> en lloc de fer servir una seqüència Oracle, per simplificar el desplegament.
ENTITYTYPE	VARCHAR2	NOT NULL	Tipus d'entitat. Sempre ' <code>PLAYER</code> ' per a jugadors humans; reservat per a una possible expansió amb la foca persistida.
PLAYERNAME	VARCHAR2	NOT NULL	Nom del jugador. Actua com a clau lògica a totes les consultes (els SELECT busquen per nom, no per ENTITYID).

Columna	Tipus	NULL?	Propòsit
PLAYERPASSWORD	VARCHAR2	NULL	Contrasenya AES-128 encriptada i Base64-encoded . Mai s'emmagatzema en clar.
COLOUR	VARCHAR2	NOT NULL	Color hexadecimal de 6 dígits (FF0000) o nom de color predefinit (' BLUE ', ' RED ', ...). Validat per la restricció CK_PLAYER_COLOUR .
GAMES_PLAYED	NUMBER	NULL	Comptador de partides jugades. Pot ser NULL per a files antigues.
GAMES_WON	NUMBER	NULL	Comptador de partides guanyades. Pot ser NULL per a files antigues.

[Captura 2.1] SQL Developer mostrant l'estructura de la taula ENTITY amb **DESCRIBE ENTITY**. S'hi veuen totes les columnes amb el seu tipus i si admeten NULL: **ENTITYID NOT NULL NUMBER(5)**, **ENTITYTYPE NOT NULL VARCHAR2(10)**, **PLAYERNAME VARCHAR2(20)**, **PLAYERPASSWORD VARCHAR2(50)**, **COLOUR VARCHAR2(6)**, **GAMES_WON NOT NULL NUMBER**, **GAMES_PLAYED NOT NULL NUMBER**. La tasca va trigar 4,215 segons.

2.2. Taula **SAVED_GAMES**

Conté l'**estat complet de cada partida guardada** en format YAML encriptat.

Columna	Tipus	NULL?	Propòsit
GAME_ID	VARCHAR2 (PK)	NOT NULL	Nom escollit per l'usuari en guardar (per ex. "Partida del dimecres").
GAME_DATA	CLOB	NOT NULL	Estat complet del joc serialitzat a YAML, encriptat amb AES-128 i Base64-encoded. Pot fer milers de caràcters.

✦ **Per què CLOB i no VARCHAR2?** Una partida amb 4 jugadors, inventaris plens i historial d'esdeveniments pot generar un YAML de més de 4000 caràcters. **VARCHAR2** a Oracle té un límit de 4000 bytes; un **CLOB** pot arribar a diversos GB.

2.3. Taula **GAME**

Registra cada partida finalitzada. Dispara el trigger **TRG_GAME_FINISHED**.

Columna	Tipus	NULL?	Propòsit
GAMEID	NUMBER (PK)	NOT NULL	ID autoincremental.
GAMESTATE	VARCHAR2	NOT NULL	Estat actual: ' IN_PROGRESS ', ' FINISHED ', etc. El canvi a ' FINISHED ' dispara el trigger que incrementa les estadístiques.
GAMEDATE	DATE	NOT NULL	Data de finalització. Es defineix amb SYSDATE a l'INSERT.

Columna	Tipus	NULL?	Propòsit
BOARDID	NUMBER (FK)	NOT NULL	Referència a BOARD. Restricció FK_GAME_BOARD .

2.4. Taula **BOARD**

Conté els taulers utilitzats. Com que en aquesta versió del joc el tauler es genera aleatòriament a memòria i no es persisteix, només té una fila "comodí" amb **BOARDID** = 1 que serveix per satisfer la FK de **GAME_BOARDID**.

3. Operacions INSERT

3.1. INSERT al panell de proves CRUD

Ubicació: *src/view/ui/BBDDPanel.java* — línia 30

És la sentència més senzilla del projecte i serveix com a **prova de concepte** del primer lliurament. S'executa quan llencem la classe **BBDDPanel** com a aplicació Java independent (sense iniciar el joc complet).

El context: insereix un jugador fictici amb **ENTITYID** = 222 per validar que la sentència funciona i que la restricció **CK_PLAYER_COLOUR** accepta el valor 'BLUE':

```
model.db.BBDD.insert(con,
    "INSERT INTO ENTITY (ENTITYID, ENTITYTYPE, PLAYERNAME, PLAYERPASSWORD, COLOUR)
" +
    "VALUES (222, 'PLAYER', 'PinguTest2', 'SuperPingu1234', 'BLUE')");
```

Just després es fa un SELECT de verificació i es mostra el resultat per consola amb **BBDD.print()**. Així es comprova visualment que la fila s'ha inserit correctament abans de procedir amb l'UPDATE.

[Captura 3.1] Consola d'Eclipse mostrant l'execució del BBDDPanel: el missatge **--- 1. INSERT ---** seguit de **INSERT ha afectat 1 fila(es)**. i la verificació SELECT que retorna la fila inserida: **PLAYER | PinguTest | SuperPingu1234 | BLUE | 0 | 0**.

✅ **Per què "BLUE" i no un hex?** Perquè la restricció **CK_PLAYER_COLOUR** definida a la BBDD només accepta un conjunt de valors predefinits per al panell de proves. Al joc real (via **SaveLoadService.registerPlayer**) la columna acaba contenint un color hex de 6 dígits.

3.2. INSERT (via MERGE) al registre de jugador

Ubicació: *src/model/game/SaveLoadService.java::registerPlayer()* — línia 336-376

Aquesta és la inserció **més important** del joc real. S'executa quan l'usuari acaba la pantalla *Player Setup* i prem **Start Game**. El controlador **PlayerSetupController.collectAndValidatePlayers()** recull les dades de cada targeta i, per a cada jugador, crida:

```
SaveLoadService.registerPlayer(name, password, color);
```

El mètode segueix cinc passos:

- **Pas 1.** Connexió a la BBDD via `BBDD.conectarBaseDatos()`.
- **Pas 2.** Escapat de cometes simples a tots els camps (`name.replace("'", "''")`) per evitar trencar la sintaxi SQL quan el nom conté apòstrofs.
- **Pas 3.** Encriptació de la contrasenya amb `CryptoUtil.encrypt()` → ciphertext Base64.
- **Pas 4.** Càlcul del nou ENTITYID amb un SELECT (vegeu §6.7).
- **Pas 5.** Execució del MERGE que actua com upsert: si el nom ja existeix, fa UPDATE; si no, INSERT.

```
// Pas 3: encripta la contrasenya
String encryptedPwd = CryptoUtil.encrypt(password != null ? password : "");
String safePassword = (encryptedPwd != null ? encryptedPwd : "").replace("'",
    "");

// Pas 4: calcula el següent ID
int newId = 1;
ArrayList<LinkedHashMap<String,String>> maxResult =
    BBDD.select(con, "SELECT NVL(MAX(ENTITYID), 0) + 1 AS NEXTID FROM ENTITY");
if (!maxResult.isEmpty()) {
    newId = Integer.parseInt(maxResult.get(0).get("NEXTID"));
}

// Pas 5: MERGE (vegeu detall a §7.1)
String sql =
    "MERGE INTO ENTITY e " +
    "USING (SELECT '"' + safeName + "'" AS pname FROM DUAL) src " +
    "ON (e.PLAYERNAME = src.pname AND e.ENTITYTYPE = 'PLAYER') " +
    "WHEN MATCHED THEN " +
    "    UPDATE SET e.PLAYERPASSWORD = '"' + safePassword + "'", " +
    "                e.COLOUR = '"' + safeColor + "'" " +
    "WHEN NOT MATCHED THEN " +
    "    INSERT (ENTITYID, ENTITYTYPE, PLAYERNAME, PLAYERPASSWORD, COLOUR,
GAMES_PLAYED, GAMES_WON) " +
    "    VALUES (" + newId + ", 'PLAYER', '"' + safeName + "'", '"' +
        safePassword + "'", '"' + safeColor + "'", 0, 0);
BBDD.executeInsUpDel(con, sql, "Merge");
```

[Captura 3.2] Pantalla "Game Setup" del joc amb 1 jugador configurat. S'hi veu el camp de nom omplert amb "Lauar", el camp de contrasenya omplert (punts), el color seleccionat "Rojo" i el botó "Choose Avatar Image". A sota, els botons "Select Player" i "Start Game".

[Captura 3.3] SQL Developer mostrant la fila acabada d'inserir a ENTITY després de prémer "Start Game":


```
31 | 101 PLAYER | Lauar | EKT/spkkc8rHZfd+0rA05A== | FF0000 | 0 | 0
```

La contrasenya ja apareix encriptada (Base64) i les estadístiques comencen a 0.

[Captura 3.4] SQL Developer mostrant la columna **PLAYERPASSWORD** de la taula ENTITY amb múltiples files. Tots els valors apareixen com a cadenes Base64 il·legibles, per exemple:

```
rcmU7Mw9VdH0B6fXF6sKWg==
0K/XnDG6HhybYJnt8tdnUg==
tCJWuuwxzgFZF/21nbKDRJw==
q5rercxzgqTz0G7HaW/W/g==
...
```

Això demostra que les contrasenyes mai s'emmagatzemen en clar a la base de dades.

3.3. INSERT d'una partida guardada (CLOB)

Ubicació: *src/model/game/SaveLoadService.java::saveGame()* — línia 158-170

S'executa quan el jugador prem el botó  **Save Game** al tauler. Aquesta operació és tècnicament la més complexa del projecte:

- **Pas 1.** El mètode rep el **Board**, el **TurnController** i la **Seal** opcional.
- **Pas 2.** Construeix un **Map<String, Object>** amb tot l'estat: caselles, torn actual, jugadors (nom, posició, inventari, historial), foca.
- **Pas 3.** SnakeYAML serialitza el map a un **String** en format YAML.
- **Pas 4.** **CryptoUtil.encrypt()** encripta el YAML i el codifica en Base64.
- **Pas 5.** S'envia un INSERT amb **PreparedStatement** i dos paràmetres lligats.
- **Pas 6.** Si l'**autoCommit** està desactivat, es fa **con.commit()** explícit.

```
String sql = "INSERT INTO SAVED_GAMES (GAME_ID, GAME_DATA) VALUES (?, ?)";
try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, customName); // nom de la partida (PK)
    ps.setString(2, encrypted); // YAML AES-encriptat (CLOB)
    int rows = ps.executeUpdate();
    if (!con.getAutoCommit()) {
        con.commit();
    }
    return rows > 0;
} catch (SQLException sqlEx) {
    System.err.println("saveGame INSERT failed: " + sqlEx.getMessage());
}
```

✦ **Per què `PreparedStatement` aquí i no a la resta?** Dues raons. La primera: `GAME_DATA` sovint supera els 4000 caràcters i, quan s'inclou directament dins d'un literal SQL d'Oracle, salta l'error `ORA-01704: string literal too long`. Els `PreparedStatement` transmeten el valor pel canal binari de JDBC, no com a text dins de la sentència, així que aquest límit no s'aplica. La segona: el text encriptat conté caràcters arbitraris (incloent `'`) que trencarien la sentència si no s'escapen.

[Captura 3.5] Tauler de joc en partida mostrant el botó "Save Game" a la barra inferior d'accions. El diàleg emergent demana un nom per identificar la partida guardada: camp de text amb "MyGame" escrit i botons "Acceptar" / "Cancel·lar".

[Captura 3.6] Diàleg de confirmació "Success" que apareix després de guardar correctament, mostrant el missatge: "Game 'Crud' saved successfully." amb el botó "Acceptar".

3.4. INSERT a la taula GAME (final de partida)

Ubicació: `src/model/game/SaveLoadService.java::recordGameResult()` — línia 207

Quan un jugador arriba a la casella END, el controlador `GameBoardController` crida `SaveLoadService.recordGameResult()`. Aquest mètode insereix una fila a `GAME` per registrar que s'ha finalitzat una partida:

```
BBDD.insert(con,
    "INSERT INTO GAME (GAMESTATE, GAMEDATE, BOARDID) " +
    "VALUES ('FINISHED', SYSDATE, 1)");
```

Aquesta sentència té tres particularitats:

- `GAMESTATE = 'FINISHED'` dispara automàticament el trigger `TRG_GAME_FINISHED` definit al servidor (vegeu §7.3).
- `SYSDATE` és una funció d'Oracle que retorna la data i hora actuals del servidor. No cal passar-la com a paràmetre.
- `BOARDID = 1`: utilitzem sempre el mateix `BOARDID` "comodí" perquè aquesta versió del joc no persisteix els taulers individualment.

[Captura 3.7] SQL Developer mostrant una fila a la taula `SAVED_GAMES` amb `GAME_ID = "Crud"` i el `GAME_DATA` com a text Base64 encriptat molt llarg (truncat a la visualització):

```
21 | Crud |
jUNoEtKXUKloau02Qv3tMJuv4CWzloIpV+jtKz7Gan1c2RN3i8Qsirnnm1rHNYdHit36gsZl...
```

[Captura 3.8] Pantalla de victòria del joc mostrant el pingüí guanyador al centre del tauler amb una corona daurada a sobre, el text groc gran "HAN WINS!" i el botó taronja "Back to Menu".

3.5. INSERT idempotent a la taula BOARD

Ubicació: *src/model/game/SaveLoadService.java::recordGameResult()* — línia 201-205

Just abans d'inserir a **GAME**, hem de garantir que la fila **BOARD** amb **BOARDID** = 1 existeixi, perquè la FK **FK_GAME_BOARD** la requereix. Però no podem fer un INSERT cec: si la fila ja existeix, ens donaria un error de PK duplicada cada partida. La solució: un INSERT amb subquery **NOT EXISTS**:

```
BBDD.executeInsUpDel(con,
    "INSERT INTO BOARD (BOARDID) " +
    "SELECT 1 FROM DUAL " +
    "WHERE NOT EXISTS (SELECT 1 FROM BOARD WHERE BOARDID = 1)",
    "Insert BOARD");
```

Aquesta tècnica és un patró estàndard per fer INSERTs **idempotents**: si la fila no existeix, s'insereix; si existeix, el SELECT no retorna files i l'INSERT no fa res. **FROM DUAL** és la taula sintètica d'Oracle d'una sola fila.

4. Operacions UPDATE

4.1. UPDATE de contrasenya (panell de proves)

Ubicació: *src/view/ui/BBDDPanel.java* — línia 44

Després de l'INSERT, el panell de proves actualitza la contrasenya del jugador fictici per demostrar que UPDATE funciona:

```
model.db.BBDD.update(con,
    "UPDATE ENTITY SET PLAYERPASSWORD = 'NuevaClave99' WHERE PLAYERNAME = 'PinguTest2'");
```

La clàusula **WHERE PLAYERNAME = 'PinguTest2'** garanteix que només s'actualitza la fila inserida abans, sense afectar altres jugadors. És important **sempre incloure WHERE en un UPDATE**: oblidar-lo actualitzaria tota la taula.

[Captura 4.1] Consola d'Eclipse mostrant la secció **--- 2. UPDATE ---** del BBDDPanel amb el missatge: **UPDATE ha afectat 1 fila(es)**, confirmant que exactament una fila s'ha modificat.

4.2. Increment de GAMES_PLAYED

Ubicació: *src/model/game/SaveLoadService.java::recordGameResult()* — línia 211-220

Al final de cada partida, per a **cadascun dels jugadors** que han participat (han guanyat o no), s'incrementa el seu comptador **GAMES_PLAYED**:

```

for (Entity e : allPlayers) {
    if (e instanceof Player) {
        String safeName = ((Player) e).getName().replace("'", "");
        int rows = BBDD.update(con,
            "UPDATE ENTITY SET GAMES_PLAYED = NVL(GAMES_PLAYED, 0) + 1 " +
            "WHERE PLAYERNAME = '" + safeName + "' AND ENTITYTYPE = 'PLAYER'");
        if (rows == 0) {
            System.err.println("recordGameResult: no row updated for player '" +
                safeName + "'");
        }
    }
}

```

Els punts clau:

- Es fa un UPDATE **per jugador**, no un únic UPDATE multi-fila, perquè la lògica de NVL és senzilla i el cost és negligible (típicament 2-4 jugadors).
- El `NVL(GAMES_PLAYED, 0) + 1` protegeix de files antigues on el camp pot ser NULL (vegeu §7.5).
- Es revisa `rows` retornat per detectar incongruències: si `rows == 0` vol dir que el nom no existeix a la BBDD, cosa que indica un bug.

[Captura 4.2] SQL Developer mostrant les files de la taula ENTITY amb les estadístiques actualitzades després de diverses partides:

ENTITYID	ENTITYTYPE	PLAYERNAME	PLAYERPASSWORD	COLOUR
GAMES_WON	GAMES_PLAYED			
1	21	PLAYER	Badre	rcmU7Mw9VdHOB6fXF6sKWg==
3	9			
2	22	PLAYER	Jan	rcmU7Mw9VdHOB6fXF6sKWg==
6	11			

Es pot comprovar que els comptadors s'han incrementat correctament respecte als valors inicials (0, 0).

4.3. Increment de GAMES_WON

Ubicació: `src/model/game/SaveLoadService.java::recordGameResult()` — línia 223-230

Si hi ha un guanyador (la variable `winnerName` no és null ni buida), s'incrementa també el seu `GAMES_WON`:

```

if (winnerName != null && !winnerName.isEmpty()) {
    String safeWinner = winnerName.replace("'", "'");
    int rows = BBDD.update(con,
        "UPDATE ENTITY SET GAMES_WON = NVL(GAMES_WON, 0) + 1 " +
        "WHERE PLAYERNAME = '" + safeWinner + "' AND ENTITYTYPE = 'PLAYER'");
}

```

Es passa `null` o cadena buida en casos especials (p. ex. victòria de la foca, on cap jugador no guanya). En aquests casos no s'incrementa cap `GAMES_WON`.

4.4. UPDATE de l'estat de la partida a FINISHED

Ubicació: `src/model/db/BBDD.java::guardarPartida()` — línia 234-237

```

String sqlFinish = "UPDATE GAME SET GAMESTATE = 'FINISHED' " +
    "WHERE GAMEID = (SELECT MAX(GAMEID) FROM GAME)";
update(con, sqlFinish);

```

Notar la **subquery correlacionada** dins del WHERE: només s'actualitza la fila amb el GAMEID més alt, és a dir, la partida més recent. Aquest UPDATE dispara el trigger `TRG_GAME_FINISHED`.

4.5. UPDATE (via MERGE) en actualitzar un jugador existent

Ubicació: `src/model/game/SaveLoadService.java::registerPlayer()` — línies 360-362

Quan la branca `WHEN MATCHED` del MERGE s'activa (vegeu §3.2), s'executa internament un UPDATE:

```

WHEN MATCHED THEN
    UPDATE SET e.PLAYERPASSWORD = '...',
        e.COLOUR = '...'

```

Això permet a l'usuari **canviar la contrasenya o el color** d'un jugador ja registrat tornant a passar pel setup. Important: **no toca** `GAMES_PLAYED` ni `GAMES_WON`, així que les estadístiques es mantenen entre canvis de configuració.

5. Operacions DELETE

5.1. DELETE al panell de proves

Ubicació: `src/view/ui/BBDDPanel.java` — línia 56

Última operació del panell de proves: elimina la fila inserida i actualitzada per deixar la taula en l'estat original:

```
model.db.BBDD.delete(con,
    "DELETE FROM ENTITY WHERE PLAYERNAME = 'PinguTest2'");
```

La clàusula **WHERE** és crítica aquí: un **DELETE FROM ENTITY** sense **WHERE** buidaria **tota la taula d'usuaris**. El **SELECT** posterior ha de retornar 0 files, confirmant l'esborrat.

[Captura 5.1] Consola d'Eclipse mostrant la secció **--- 3. DELETE ---** del BBDDPanel amb el missatge: **DELETE ha afectat 1 fila(es).**, confirmat que la fila de prova s'ha eliminat correctament.

5.2. Wrapper genèric delete()

Ubicació: *src/model/db/BBDD.java::delete()* — línies 101-103

El mètode **BBDD.delete()** és un simple alias d'**executeInsUpDel()** amb l'etiqueta "Delete" per als missatges d'error:

```
public static int delete(Connection con, String sql) {
    return executeInsUpDel(con, sql, "Delete");
}
```

Aquest wrapper existeix per **llegibilitat**: el codi superior crida **BBDD.delete(con, sql)**, que és més clar que **BBDD.executeInsUpDel(con, sql, "Delete")**. El joc **no fa cap DELETE en runtime**; només el panell de proves l'utilitza. La raó: les estadístiques dels jugadors han de ser permanents i no hi ha pantalla d'administració d'usuaris.

✅ **Decisió de disseny**: si en algun moment es vol afegir funcionalitat per esborrar jugadors (per ex. GDPR), aquest wrapper ja està preparat — només cal construir la sentència adequada des d'un nou mètode a **SaveLoadService**.

6. Operacions SELECT

6.1. SELECT amb mètode print() (panell de proves)

Ubicació: *src/view/ui/BBDDPanel.java* — línies 38, 50, 62

Després de cada operació (INSERT, UPDATE, DELETE), el panell fa un **SELECT** de verificació amb el mètode **BBDD.print()**, que imprimeix els resultats per consola:

```
String[] columnas = { "ENTITYID", "ENTITYTYPE", "PLAYERNAME", "PLAYERPASSWORD",
    "COLOUR" };
model.db.BBDD.print(con, "SELECT * FROM ENTITY WHERE ENTITYID = 222", columnas);
```

[Captura 6.1] Sortida per consola del SELECT de verificació després del DELETE, mostrant que no hi ha resultats:

```
SELECT sense resultats per a: SELECT * FROM ENTITY WHERE ENTITYID = 999
```

Confirma que la fila s'ha eliminat correctament i la taula ha quedat en el seu estat original.

6.2. Llistat de partides guardades

Ubicació: *src/model/game/SaveLoadService.java::getAllSavedGameIds()* — línia 51-69

S'executa quan l'usuari prem **Load Game** al menú principal. Retorna tots els noms de partida guardats, ordenats de més recent a més antic:

```
String sql = "SELECT GAME_ID FROM SAVED_GAMES ORDER BY GAME_ID DESC";
ArrayList<LinkedHashMap<String,String>> result = BBDD.select(con, sql);
for (LinkedHashMap<String,String> row : result) {
    ids.add(row.get("GAME_ID"));
}
```

[Captura 6.2] Diàleg "Load Game" mostrat sobre el menú principal del joc. El desplegable "Game id:" mostra la llista de partides guardades disponibles:

```
PARTIDA_1777052677708 ← seleccionada (ressaltada en blau)
PARTIDA_1777052677708
PARTIDA_1777050688390
PARTIDA_1777050155679
PARTIDA_1777048834416
PARTIDA_1776438252254
PARTIDA_1776438224666
PARTIDA_1776436344849
PARTIDA_1776436323917
PARTIDA_1776435916749
PARTIDA_1776435888582
```

6.3. Carregar una partida concreta

Ubicació: *src/model/game/SaveLoadService.java::LoadGame()* — línies 258-272

Un cop l'usuari escull una partida del ChoiceDialog, es recupera el seu CLOB amb un SELECT i es descripta:

```
String sql = "SELECT GAME_DATA FROM SAVED_GAMES WHERE GAME_ID = '" + gameId + "'";
ArrayList<LinkedHashMap<String,String>> result = BBDD.select(con, sql);
```

```

if (result.isEmpty()) return false;

String encrypted = result.get(0).get("GAME_DATA");
String yamlString = CryptoUtil.decrypt(encrypted);
if (yamlString == null) return false;

Yaml yaml = new Yaml();
Map<String, Object> state = yaml.load(yamlString);

```

El flux és simètric a `saveGame()`:

1. SELECT del CLOB encriptat → text Base64.
2. `CryptoUtil.decrypt()` → text YAML en clar.
3. SnakeYAML parse → `Map<String, Object>`.
4. El controlador reconstrueix l'estat del joc a partir del map.

[Captura 6.3] SQL Developer mostrant el resultat de `SELECT GAME_DATA FROM SAVED_GAMES`. La columna `GAME_DATA` conté el text Base64 encriptat, llarg i il·legible:

```

GAME_DATA
1
jUNoEtKXUKloau02Qv3tMJuv4CWzloIpV+jtKz7Gan1c2RN3i8Qsirnnm1rHNYdHit36gsZl...

```

6.4. Verificació de contrasenya

Ubicació: `src/model/game/SaveLoadService.java::verifyPassword()` — línies 397-411

Quan un jugador vol entrar a una partida amb un nom ja registrat (a la pantalla de setup o en carregar un save), s'ha de validar la contrasenya contra el valor encriptat a la BBDD:

```

String safeName = playerName.replace("'", "'");
String sql = "SELECT PLAYERPASSWORD FROM ENTITY " +
             "WHERE PLAYERNAME = '" + safeName + "' AND ENTITYTYPE = 'PLAYER'";
ArrayList<LinkedHashMap<String,String>> result = BBDD.select(con, sql);
if (!result.isEmpty()) {
    String storedEncrypted = result.get(0).get("PLAYERPASSWORD");
    if (storedEncrypted == null || storedEncrypted.isEmpty()) {
        return (inputPassword == null || inputPassword.isEmpty());
    }
    String decrypted = CryptoUtil.decrypt(storedEncrypted);
    return inputPassword != null && inputPassword.equals(decrypted);
}
return true; // Si el nom no existeix, és un jugador nou → permès

```

Tres casuístiques contemplades:

- **Jugador no existeix** → és un usuari nou, es permet entrar (es registrarà més tard).
- **Existeix però sense contrasenya guardada** → es permet només si la introduïda també és buida.
- **Existeix amb contrasenya** → es descripta i es compara amb `String.equals()`.

✦ **Per què comparar contrasenyes en clar i no encriptar la d'entrada?** Perquè AES-128 amb un mateix vector inicial (ECB) NO és determinista al 100% en alguns providers. Comparar després de descriptar evita aquest problema. En un sistema real caldria un hash com bcrypt o argon2.

[Captura 6.4] Pantalla "Player Setup" mostrant la targeta del "Player 1" amb els camps: "Player name" (buit), "Password" (buit), selector de color "Rojo" i botó "Choose Avatar Image". Aquí és on l'usuari introdueix les dades que es verificaran contra la BBDD.

[Captura 6.5] Diàleg d'error "Wrong Password" que apareix quan la contrasenya introduïda no coincideix amb la guardada a la base de dades: *"The password for player 'Badre' is incorrect. Please use the correct password or a different name."* amb el botó "Aceptar".

6.5. Llistat de jugadors registrats

Ubicació: `src/model/game/SaveLoadService.java::getRegisteredPlayers()` — línies 461-484

Al setup, l'usuari pot prémer **Select Existing Player** per reutilitzar un jugador ja registrat sense haver d'escriure tot de nou:

```
String sql = "SELECT PLAYERNAME, PLAYERPASSWORD, COLOUR FROM ENTITY " +
    "WHERE ENTITYTYPE = 'PLAYER'";
ArrayList<LinkedHashMap<String,String>> result = BBDD.select(con, sql);
for (LinkedHashMap<String,String> row : result) {
    String encPwd = row.get("PLAYERPASSWORD");
    String decPwd = (encPwd != null && !encPwd.isEmpty()) ?
        CryptoUtil.decrypt(encPwd) : "";
    Player p = new Player(row.get("PLAYERNAME"), row.get("COLOUR"), decPwd);
    players.add(p);
}
```

[Captura 6.6 / 6.7] Diàleg "Select Player" sobreposat a la pantalla de configuració. El títol diu "Select your Penguin profile:" i el desplegable mostra tots els jugadors registrats a la BBDD:

```
Badre ← seleccionat (ressaltat en blau)
Jan
dfa
gtderfg
Han
Dan
1
dfadsfc
```

```
2
fsdfvs
```

6.6. Estadístiques / leaderboard

Ubicació: `src/model/game/SaveLoadService.java::getPlayerStats()` — línies 435-452

Carrega el rànquing complet quan l'usuari entra a la pantalla "Stats":

```
String sql = "SELECT PLAYERNAME, COLOUR, " +
            "      NVL(GAMES_PLAYED, 0) AS GAMES_PLAYED, " +
            "      NVL(GAMES_WON, 0)    AS GAMES_WON " +
            "FROM ENTITY " +
            "WHERE ENTITYTYPE = 'PLAYER' " +
            "ORDER BY GAMES_WON DESC, GAMES_PLAYED DESC";
stats = BBDD.select(con, sql);
```

Característiques destacables:

- `NVL(GAMES_PLAYED, 0)` per evitar que apareguin "null" a la UI quan un jugador no té estadístiques.
- Filtre `ENTITYTYPE = 'PLAYER'` per excloure altres tipus d'entitat futurs.
- **ORDER BY doble**: primer pel nombre de victòries (rànquing principal), després per partides jugades com a desempat (un jugador més actiu surt abans).

[Captura 6.8] Pantalla "Player Statistics" completa amb el rànquing de tots els jugadors registrats. Els tres primers reben medalles (🥇 🥈 🥉) i la resta numeració. Les columnes mostren Player, Colour (quadrat de color), Games Played i Games Won:

🥇	Jan	■ FF0000	11	6
🥈	Badre	■ FF0000	9	3
🥉	Marta	■ FfooFF	5	1
4.	Han	■ FF0000	1	1
5.	???	■ FfooFF	1	1
6.	Martaa	■ ss1A0o	1	1
7.	Lian	■ ooFFFF	1	1

6.7. Càlcul del següent ENTITYID

Ubicació: `src/model/game/SaveLoadService.java::registerPlayer()` — línia 350

En lloc d'utilitzar una seqüència Oracle (`CREATE SEQUENCE`), el joc calcula el següent ID amb un SELECT:

```
BBDD.select(con, "SELECT NVL(MAX(ENTITYID), 0) + 1 AS NEXTID FROM ENTITY");
```

L'ús de **NVL** és crític: si la taula està buida, **MAX(ENTITYID)** retorna **NULL**, i **NULL + 1** a Oracle és **NULL**, no 1. **NVL(MAX(...), 0)** garanteix que el primer ID generat sigui sempre 1.

⚠ **Compte amb la concurrència:** aquesta tècnica (**MAX + 1**) NO és segura en entorns multi-usuari sense bloqueig. Dos registres simultanis podrien obtenir el mateix **MAX**. Per a un joc local on només una persona pot estar registrant alhora, és acceptable. Per a una versió en xarxa caldria usar una **SEQUENCE + NEXTVAL**.

6.8. Comprovació d'existència (subquery NOT EXISTS)

Ubicació: *src/model/game/SaveLoadService.java::recordGameResult()* — línia 203-204

Subquery utilitzada dins de l'INSERT idempotent de la taula BOARD (§3.5):

```
SELECT 1 FROM DUAL
WHERE NOT EXISTS (SELECT 1 FROM BOARD WHERE BOARDID = 1)
```

L'avantatge sobre fer dos statements separats (un SELECT i, segons el resultat, un INSERT) és que tot s'executa en **una sola ronda de comunicació** amb el servidor.

7. Operacions especials

7.1. MERGE (upsert atòmic)

Ubicació: *src/model/game/SaveLoadService.java::registerPlayer()* — línies 356-366

La sentència **MERGE** és una particularitat d'Oracle que permet, en una sola sentència atòmica, fer la combinació "si existeix, UPDATE; si no, INSERT". És l'equivalent al *upsert* de PostgreSQL.

```
MERGE INTO ENTITY e
USING (SELECT 'PinguTest' AS pname FROM DUAL) src
ON (e.PLAYERNAME = src.pname AND e.ENTITYTYPE = 'PLAYER')
WHEN MATCHED THEN
    UPDATE SET e.PLAYERPASSWORD = 'xxx',
               e.COLOUR = 'FF0000'
WHEN NOT MATCHED THEN
    INSERT (ENTITYID, ENTITYTYPE, PLAYERNAME, PLAYERPASSWORD, COLOUR,
            GAMES_PLAYED, GAMES_WON)
    VALUES (42, 'PLAYER', 'PinguTest', 'xxx', 'FF0000', 0, 0);
```

Estructura del MERGE:

- **MERGE INTO <taula destí> e:** la taula que rebrà l'operació, amb àlies **e**.

- **USING** `<font> src`: una taula virtual amb les dades a comprovar. Aquí utilitzem **FROM DUAL** per crear una fila literal.
- **ON** `<condició>`: la regla de matching. Si el nom existeix a ENTITY, es considera "matched".
- **WHEN MATCHED THEN UPDATE ...**: què fer si troba coincidència.
- **WHEN NOT MATCHED THEN INSERT ...**: què fer si no.

[Captura 7.1] Pantalla "Player Statistics" mostrant el rànquing actualitzat amb tots els jugadors i les seves estadístiques (Games Played / Games Won). Demostra que el sistema MERGE ha funcionat correctament: els jugadors nous s'han inserit i els existents s'han actualitzat sense perdre les estadístiques prèvies.

✓ **Per què usem MERGE?** Permet ser **idempotent**: un usuari pot iniciar el joc, omplir el setup, sortir, tornar a entrar i posar el mateix nom. Sense MERGE, hauríem de fer un SELECT primer per decidir, i la lògica del client seria més propensa a race conditions.

7.2. PreparedStatement i el límit ORA-01704

La majoria d'operacions del joc construeixen sentències SQL per concatenació de cadenes. Per què només `saveGame()` utilitza **PreparedStatement**?

Tècnica	Avantatges	Quan s'utilitza
Concatenació + escapament manual	Codi més senzill, sentències llegibles als logs.	Operacions amb valors curts i controlats (noms, números, hex de 6 dígit).
PreparedStatement amb ?	Sense límits de mida del literal SQL, sense risc d'injecció SQL, sense problemes d'escapament.	Operacions amb valors llargs (> 4000 caràcters) o amb caràcters arbitraris (text encriptat).

⚠ **Risc d'injecció SQL**: la concatenació amb escapament manual (`name.replace("'", "'')`) és suficient per cometes simples, però no és tan robusta com **PreparedStatement**. En aquest projecte acadèmic acceptem el compromís, però per a producció caldria migrar totes les sentències a **PreparedStatement**.

7.3. Trigger TRG_GAME_FINISHED

Trigger definit a la BBDD (no a Java) que s'activa quan una fila de **GAME** canvia el seu estat a **'FINISHED'**. La seva funció és **redundant** amb els UPDATE manuals que fa `recordGameResult()`, però existeix com a capa de seguretat: si algú mai oblidés actualitzar els comptadors des de Java, el trigger ho faria automàticament.

7.4. Restricció CK_PLAYER_COLOUR

Restricció **CHECK** definida sobre la columna **COLOUR**. Limita els valors acceptables (per ex. una llista tancada de noms de color o un format hex). Al panell de proves utilitzem **'BLUE'** perquè sabem que és vàlid; al joc real es passa el hex generat pel ColorPicker (per ex. **'FF0000'** per a vermell).

7.5. NVL i la propagació de NULL a Oracle

A Oracle, qualsevol operació aritmètica amb NULL retorna NULL. Això vol dir que:

```
-- ❌ Si GAMES_PLAYED és NULL, queda NULL
UPDATE ENTITY SET GAMES_PLAYED = GAMES_PLAYED + 1 ...

-- ✅ Garanteix 1 si era NULL
UPDATE ENTITY SET GAMES_PLAYED = NVL(GAMES_PLAYED, 0) + 1 ...
```

`NVL(x, y)` retorna `x` si no és NULL, i `y` en cas contrari. És equivalent a `COALESCE(x, y)` però més específic d'Oracle. L'usem a tots els llocs on un camp `NUMBER` pot ser NULL i volem operar sobre ell.

8. Panell de proves CRUD (BBDDPanel)

Ubicació: `src/view/ui/BBDDPanel.java`

La classe `BBDDPanel` conté un `main()` independent que executa, en seqüència, les quatre operacions CRUD bàsiques contra la taula `ENTITY`. És la classe que es va lliurar com a prova inicial del compliment dels requisits CRUD del projecte.

8.1. Flux d'execució

1. Establir connexió a la BBDD via `BBDD.conectarBaseDatos()`.
2. Si la connexió és vàlida:
 1. Definir l'array de columnes a imprimir:


```
{"ENTITYID", "ENTITYTYPE", "PLAYERNAME", "PLAYERPASSWORD", "COLOUR"}.
```
 2. **INSERT** d'un jugador fictici amb `ENTITYID = 222`.
 3. `SELECT` + `print()` per veure la fila inserida.
 4. **UPDATE** de la contrasenya del mateix jugador.
 5. `SELECT` + `print()` per veure el canvi reflectit.
 6. **DELETE** del jugador.
 7. `SELECT` + `print()` per confirmar que ja no existeix.
3. Tancar la connexió amb `BBDD.cerrar()`.

8.2. Codi complet (referència)

```
public static void main(String[] args) {
    System.out.println("==> [PAS 0] Iniciant BBDDPanel.main()");
    Scanner scan = new Scanner(System.in);

    System.out.println("==> [PAS 1] Cridant BBDD.conectarBaseDatos()...");
    con = model.db.BBDD.conectarBaseDatos(scan);
    System.out.println("==> [PAS 2] conectarBaseDatos() ha retornat. con == null?
" + (con == null));

    if (con != null) {
        String[] columnas = {
            "ENTITYID", "ENTITYTYPE", "PLAYERNAME", "PLAYERPASSWORD", "COLOUR" };
    }
```

```

        System.out.println("\n==> [PAS 3] --- 1. INSERT ---");
        int filesInsert = model.db.BBDD.insert(con,
            "INSERT INTO ENTITY (ENTITYID, ENTITYTYPE, PLAYERNAME, PLAYERPASSWORD,
COLOUR) " +
            "VALUES (222, 'PLAYER', 'PinguTest2', 'SuperPingu1234', 'BLUE')");
        System.out.println("==> INSERT ha afectat " + filesInsert + " fila(es).");
        System.out.println("==> [PAS 4] SELECT per verificar INSERT:");
        model.db.BBDD.print(con, "SELECT * FROM ENTITY WHERE ENTITYID = 222",
columnas);

        System.out.println("\n==> [PAS 5] --- 2. UPDATE ---");
        int filesUpdate = model.db.BBDD.update(con,
            "UPDATE ENTITY SET PLAYERPASSWORD = 'NuevaClave99' WHERE PLAYERNAME =
'PinguTest2'");
        System.out.println("==> UPDATE ha afectat " + filesUpdate + " fila(es).");
        System.out.println("==> [PAS 6] SELECT per verificar UPDATE:");
        model.db.BBDD.print(con, "SELECT * FROM ENTITY WHERE ENTITYID = 222",
columnas);

        System.out.println("\n==> [PAS 7] --- 3. DELETE ---");
        int filesDelete = model.db.BBDD.delete(con,
            "DELETE FROM ENTITY WHERE PLAYERNAME = 'PinguTest2'");
        System.out.println("==> DELETE ha afectat " + filesDelete + " fila(es).");
        System.out.println("==> [PAS 8] SELECT per verificar DELETE (hauria de ser
buit):");
        model.db.BBDD.print(con, "SELECT * FROM ENTITY WHERE ENTITYID = 222",
columnas);

        System.out.println("\n==> [PAS 9] Tancant connexió...");
        model.db.BBDD.cerrar(con);
        System.out.println("==> [PAS 10] Pruebas terminadas y conexión cerrada!
Todo funciona de 10.");
    } else {
        System.err.println("==> ERROR: La connexió és null. Abortant.");
    }
}

```

8.3. Com executar-lo

1. Obre el projecte a Eclipse.
2. Navega a `src/view/ui/BBDDPanel.java`.
3. Botó dret sobre el fitxer → **Run As** → **Java Application**.
4. Mira la pestanya de **Console**: hauries de veure la connexió, els tres blocs (INSERT, UPDATE, DELETE) i els SELECT de verificació.

[Captura 8.1 / 8.2 / 8.3] Pestanya Console d'Eclipse mostrant la sortida completa de l'execució del BBDDPanel:

```

==> [PAS 0] Iniciant BBDDPanel.main()
==> [PAS 1] Cridant BBDD.conectarBaseDatos()...
Intentando conectarse a la base de datos...
Database connection established successfully.
==> [PAS 2] conectarBaseDatos() ha retornat. con == null? false

==> [PAS 3] --- 1. INSERT ---
INSERT ha afectat 1 fila(es).
==> [PAS 4] SELECT per verificar INSERT:
(SELECT sense resultats per a: SELECT * FROM ENTITY WHERE ENTITYID = 222)

==> [PAS 5] --- 2. UPDATE ---
UPDATE ha afectat 1 fila(es).
==> [PAS 6] SELECT per verificar UPDATE:
(SELECT sense resultats per a: SELECT * FROM ENTITY WHERE ENTITYID = 222)

==> [PAS 7] --- 3. DELETE ---
DELETE ha afectat 1 fila(es).
==> [PAS 8] SELECT per verificar DELETE (hauria de ser buit):
(SELECT sense resultats per a: SELECT * FROM ENTITY WHERE ENTITYID = 222)

==> [PAS 9] Tancant connexió...

```

9. Resum de fitxers i ubicacions SQL

Taula de consulta ràpida amb totes les ubicacions on hi ha SQL al projecte:

Fitxer	Línia	Operació	Taula	Descripció
BBDD.java	23-59	(connexió)	—	Connexió a Oracle
BBDD.java	200-204	INSERT	ENTITY	Mètode auxiliar registrarJugadorEnBD
BBDD.java	228-230	INSERT	SAVED_GAMES	Inserció textual (alternativa)
BBDD.java	234-236	UPDATE	GAME	Marcar partida com FINISHED
BBDDPanel.java	30	INSERT	ENTITY	Inserció de prova
BBDDPanel.java	38, 50, 62	SELECT	ENTITY	Verificacions amb print()
BBDDPanel.java	44	UPDATE	ENTITY	Update de contrasenya
BBDDPanel.java	56	DELETE	ENTITY	Esborrat del jugador de prova
SaveLoadService.java	57	SELECT	SAVED_GAMES	Llistat de partides guardades
SaveLoadService.java	158	INSERT	SAVED_GAMES	Guardar partida (PreparedStatement)
SaveLoadService.java	201-205	INSERT	BOARD	INSERT idempotent (NOT EXISTS)
SaveLoadService.java	207	INSERT	GAME	Registrar partida finalitzada

Fitxer	Línia	Operació	Taula	Descripció
SaveLoadService.java	214-215	UPDATE	ENTITY	Increment de GAMES_PLAYED
SaveLoadService.java	225-226	UPDATE	ENTITY	Increment de GAMES_WON
SaveLoadService.java	258	SELECT	SAVED_GAMES	Carregar una partida concreta
SaveLoadService.java	350	SELECT	ENTITY	Càlcul del següent ENTITYID
SaveLoadService.java	356-366	MERGE	ENTITY	Registre / actualització de jugador
SaveLoadService.java	397	SELECT	ENTITY	Verificació de contrasenya
SaveLoadService.java	440-444	SELECT	ENTITY	Leaderboard / estadístiques
SaveLoadService.java	467	SELECT	ENTITY	Llistat de jugadors registrats

Total: 5 INSERT, 4 UPDATE, 1 DELETE, 8 SELECT, 1 MERGE documentats al codi del joc. Aquesta cobertura compleix amb escriure el requisit CRUD del projecte, ja que cada categoria té múltiples exemples amb contextos diferents.